Wellbeing Indicator Model (Regression)

Objective

Build a machine learning regression model to predict the **composite_wellbeing_score** (0-100) from raw student features includingacademic performance, attendance, stress levels,motivation, and social factors.

Why Regression?

The wellbeing score is a continuous variable (0-100), making regression the appropriate modeling approach. We will train and compare 6 different regression models to find the best predictor.

Section 1: Environment Setup & Library Installation

In [1]:
```
# ============================================================
# SECTION 1: INSTALL AND IMPORT REQUIRED LIBRARIES
# ============================================================
# Install required packages
!pip install pandas numpy scikit-learn matplotlib seaborn xgboost

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')

# Set visualization style for professional-looking charts
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 12
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12

print("=" * 60)
print("WELLBEING INDICATOR MODEL (REGRESSION)")
print("=" * 60)
print("All libraries loaded successfully!")
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-package
s (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-p
ackages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-pac
kages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packag
es (0.13.2)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packag
es (3.2.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas) (2026.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-p
ackages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.1
2/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/di
st-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-p
ackages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dis
t-packages (from matplotlib) (26.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-pack
ages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/di
st-packages (from matplotlib) (3.3.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/di
st-packages (from xgboost) (2.30.4)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas) (1.17.0)
============================================================
WELLBEING INDICATOR MODEL (REGRESSION)
============================================================
All libraries loaded successfully!
```

Section 2: Data Loading & Initial Inspection

We load the synthetic dataset containing 2,400 records (300 students x 8 weeks).Each row represents one student's data for a specific week.

In [2]:

```
# ============================================================
# SECTION 2: LOAD AND INSPECT THE DATASET
# ============================================================

# Load the dataset
# If running in Google Colab, upload the file first or mount Google Drive
# from google.colab import files
# uploaded = files.upload()

df = pd.read_csv('student_wellbeing_dataset.csv')

# Display basic information about the dataset
print("=" * 60)
print("DATASET OVERVIEW")
print("=" * 60)
print(f"\nShape: {df.shape[0]} rows x {df.shape[1]} columns")
print(f"Memory Usage: {df.memory_usage(deep=True).sum() / 1024:.1f} KB")

# Display the first 10 rows to understand the structure
print("\n--- First 10 Rows ---")
df.head(10)


# Display column names and data types
print("\n--- Column Information ---")
print(df.dtypes)
print(f"\nTotal Columns: {len(df.columns)}")


# Check for missing values - critical for ML model training
print("\n--- Missing Values Check ---")
missing = df.isnull().sum()
print(f"Total missing values: {missing.sum()}")
if missing.sum() == 0:
    print("[OK] No missing values found. Dataset is clean!")
else:
    print("[WARNING] Missing values detected:")
    print(missing[missing > 0])


# Statistical summary of all numerical features
print("\n--- Statistical Summary ---")
df.describe().round(2)
```

```
============================================================
DATASET OVERVIEW
============================================================

Shape: 2400 rows x 26 columns
Memory Usage: 698.3 KB

--- First 10 Rows ---

--- Column Information ---
student_id                     object
grade                           int64
week                            int64
avg_exam_score                  int64
study_hours                   float64
homework_completion_rate      float64
attendance_rate               float64
absence_count                   int64
stress_level                  float64
anxiety_level                   int64
sleep_hours                   float64
fatigue_level                   int64
motivation_level                int64
confidence_level                int64
peer_interaction                int64
teacher_support                 int64
class_participation             int64
teacher_engagement_score        int64
behavior_rating                 int64
academic_stress_index         float64
attendance_stability_index    float64
performance_trend_index       float64
behavioral_score              float64
composite_wellbeing_score     float64
risk_label                     object
next_week_wellbeing_score     float64
dtype: object

Total Columns: 26

--- Missing Values Check ---
Total missing values: 0
[OK] No missing values found. Dataset is clean!

--- Statistical Summary ---
```

Out[2]:

	grade	week	avg_exam_score	study_hours	homework_completion_ra
count	2400.00	2400.00	2400.00	2400.00	2400.
mean	8.41	4.50	62.67	3.50	0.
std	1.68	2.29	11.64	1.19	0.
min	6.00	1.00	16.00	0.00	0.
25%	7.00	2.75	55.00	2.60	0.
50%	8.00	4.50	63.00	3.50	0.
75%	10.00	6.25	70.00	4.40	0.
max	11.00	8.00	100.00	6.00	1.

8 rows × 24 columns

Section 3: Exploratory Data Analysis (EDA)

We perform comprehensive EDA to understand feature distributions,correlations, and relationships before building models. This helps us identify which features are most important for predicting wellbeing.

In [3]:

```
# ============================================================
# EDA CHART 1: CORRELATION HEATMAP
# ============================================================
# Purpose: Identify which features are strongly correlated with
# composite_wellbeing_score (our target variable)

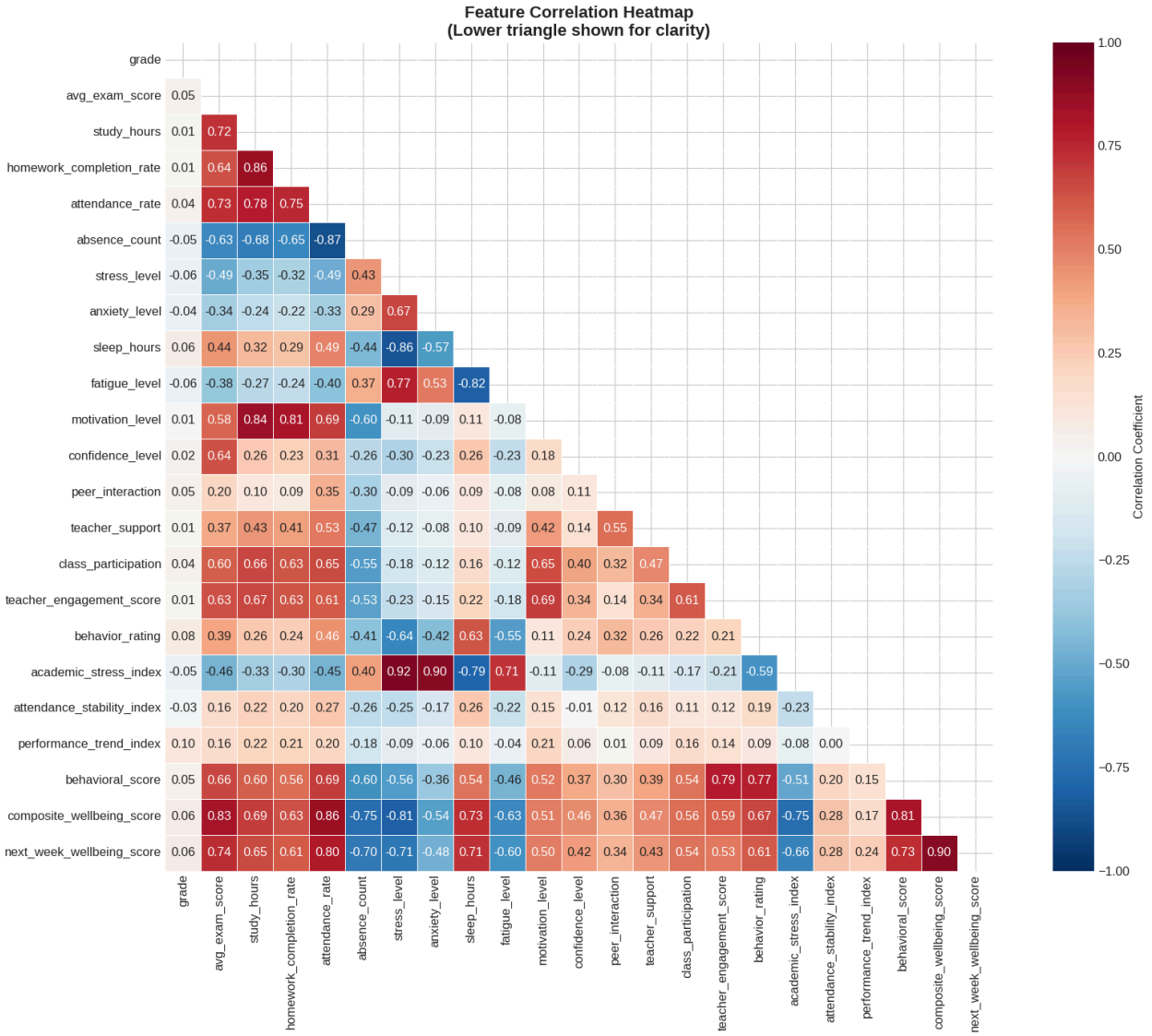
# Select only numerical columns for correlation analysis
numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
# Remove student_id-like columns if any
cols_to_exclude = ['week']  # Keep grade as it might be relevant
numerical_for_corr = [c for c in numerical_cols if c not in cols_to_exclude]

# Compute correlation matrix
corr_matrix = df[numerical_for_corr].corr()

# Create a large heatmap
fig, ax = plt.subplots(figsize=(18, 14))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))  # Mask upper triangle
sns.heatmap(
    corr_matrix, mask=mask, annot=True, fmt='.2f',
    cmap='RdBu_r', center=0, vmin=-1, vmax=1,
    square=True, linewidths=0.5,
    cbar_kws={'label': 'Correlation Coefficient'}
)
plt.title('Feature Correlation Heatmap\n(Lower triangle shown for clarity)', f
plt.tight_layout()
plt.savefig('chart1_correlation_heatmap.png', dpi=150, bbox_inches='tight')
```

```python
plt.show()

# Print top correlations with target variable
print("\n--- Top Correlations with composite_wellbeing_score ---")
target_corr = corr_matrix['composite_wellbeing_score'].drop('composite_wellbei
print("\nPositive correlations (features that INCREASE wellbeing):")
for feat, val in target_corr.head(5).items():
    print(f"  {feat}: {val:.3f}")
print("\nNegative correlations (features that DECREASE wellbeing):")
for feat, val in target_corr.tail(5).items():
    print(f"  {feat}: {val:.3f}")
```



```
--- Top Correlations with composite_wellbeing_score ---

Positive correlations (features that INCREASE wellbeing):
  next_week_wellbeing_score: 0.900
  attendance_rate: 0.857
  avg_exam_score: 0.829
  behavioral_score: 0.809
  sleep_hours: 0.728

Negative correlations (features that DECREASE wellbeing):
  anxiety_level: -0.545
  fatigue_level: -0.633
  academic_stress_index: -0.748
  absence_count: -0.749
  stress_level: -0.808
```

In [4]:

```python
# ============================================================
# EDA CHART 2: WELLBEING SCORE DISTRIBUTION
# ============================================================
# Purpose: Understand the distribution of our target variable
# A normal-like distribution is ideal for regression

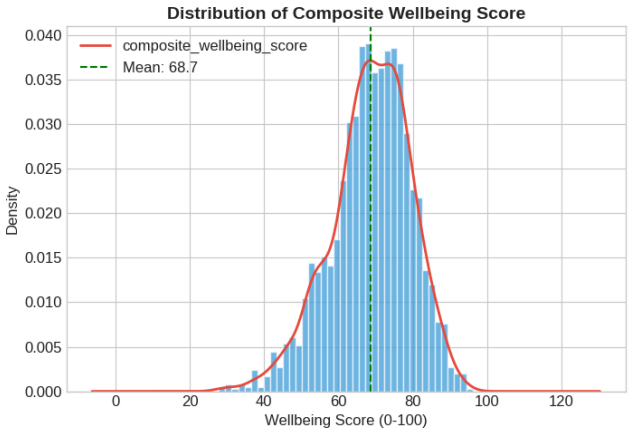
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

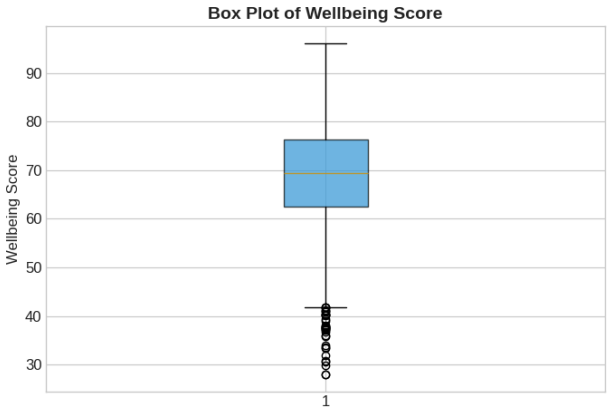
# Histogram with KDE
axes[0].hist(df['composite_wellbeing_score'], bins=40, color='#3498db',
             alpha=0.7, edgecolor='white', density=True)
df['composite_wellbeing_score'].plot.kde(ax=axes[0], color='#e74c3c', linewidt
axes[0].set_title('Distribution of Composite Wellbeing Score', fontweight='bol
axes[0].set_xlabel('Wellbeing Score (0-100)')
axes[0].set_ylabel('Density')
axes[0].axvline(df['composite_wellbeing_score'].mean(), color='green',
                linestyle='--', label=f"Mean: {df['composite_wellbeing_score']
axes[0].legend()

# Box plot
bp = axes[1].boxplot(df['composite_wellbeing_score'], vert=True, patch_artist=
                     boxprops=dict(facecolor='#3498db', alpha=0.7))
axes[1].set_title('Box Plot of Wellbeing Score', fontweight='bold')
axes[1].set_ylabel('Wellbeing Score')

plt.tight_layout()
plt.savefig('chart2_wellbeing_distribution.png', dpi=150, bbox_inches='tight')
plt.show()

# Print distribution statistics
print(f"\nWellbeing Score Statistics:")
print(f"  Mean: {df['composite_wellbeing_score'].mean():.2f}")
print(f"  Median: {df['composite_wellbeing_score'].median():.2f}")
print(f"  Std Dev: {df['composite_wellbeing_score'].std():.2f}")
print(f"  Skewness: {df['composite_wellbeing_score'].skew():.3f}")
print(f"  Kurtosis: {df['composite_wellbeing_score'].kurtosis():.3f}")
```






```
Wellbeing Score Statistics:
  Mean: 68.74
  Median: 69.50
  Std Dev: 10.75
  Skewness: -0.497
  Kurtosis: 0.280
```

In [5]:

```python
# ============================================================
# EDA CHART 3: RISK LABEL DISTRIBUTION (PIE + BAR)
# ============================================================
# Purpose: Understand the class balance in the dataset

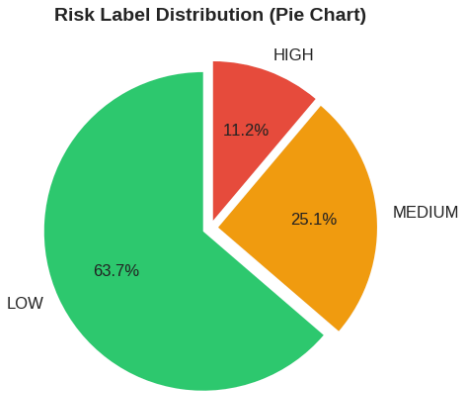
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

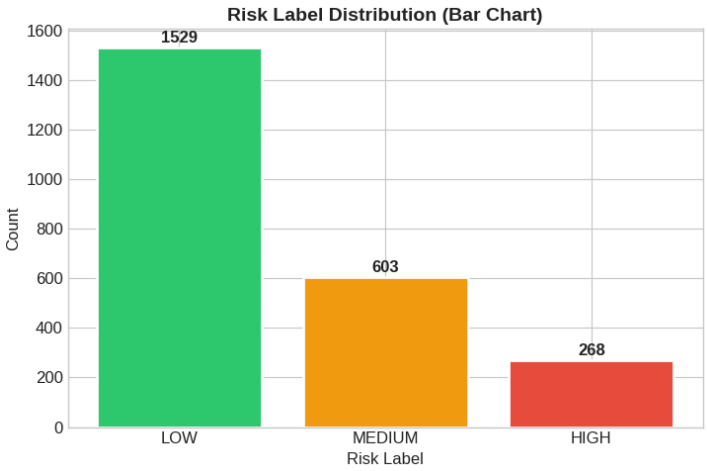
risk_counts = df['risk_label'].value_counts()
colors = {'LOW': '#2ecc71', 'MEDIUM': '#f39c12', 'HIGH': '#e74c3c'}
color_list = [colors[label] for label in risk_counts.index]

# Pie chart
axes[0].pie(risk_counts.values, labels=risk_counts.index, autopct='%1.1f%%',
            colors=color_list, startangle=90, textprops={'fontsize': 12},
            explode=[0.05] * len(risk_counts))
axes[0].set_title('Risk Label Distribution (Pie Chart)', fontweight='bold')

# Bar chart
bars = axes[1].bar(risk_counts.index, risk_counts.values, color=color_list,
                   edgecolor='white', linewidth=2)
for bar, count in zip(bars, risk_counts.values):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
                 str(count), ha='center', fontweight='bold', fontsize=12)
axes[1].set_title('Risk Label Distribution (Bar Chart)', fontweight='bold')
axes[1].set_xlabel('Risk Label')
axes[1].set_ylabel('Count')

plt.tight_layout()
plt.savefig('chart3_risk_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```





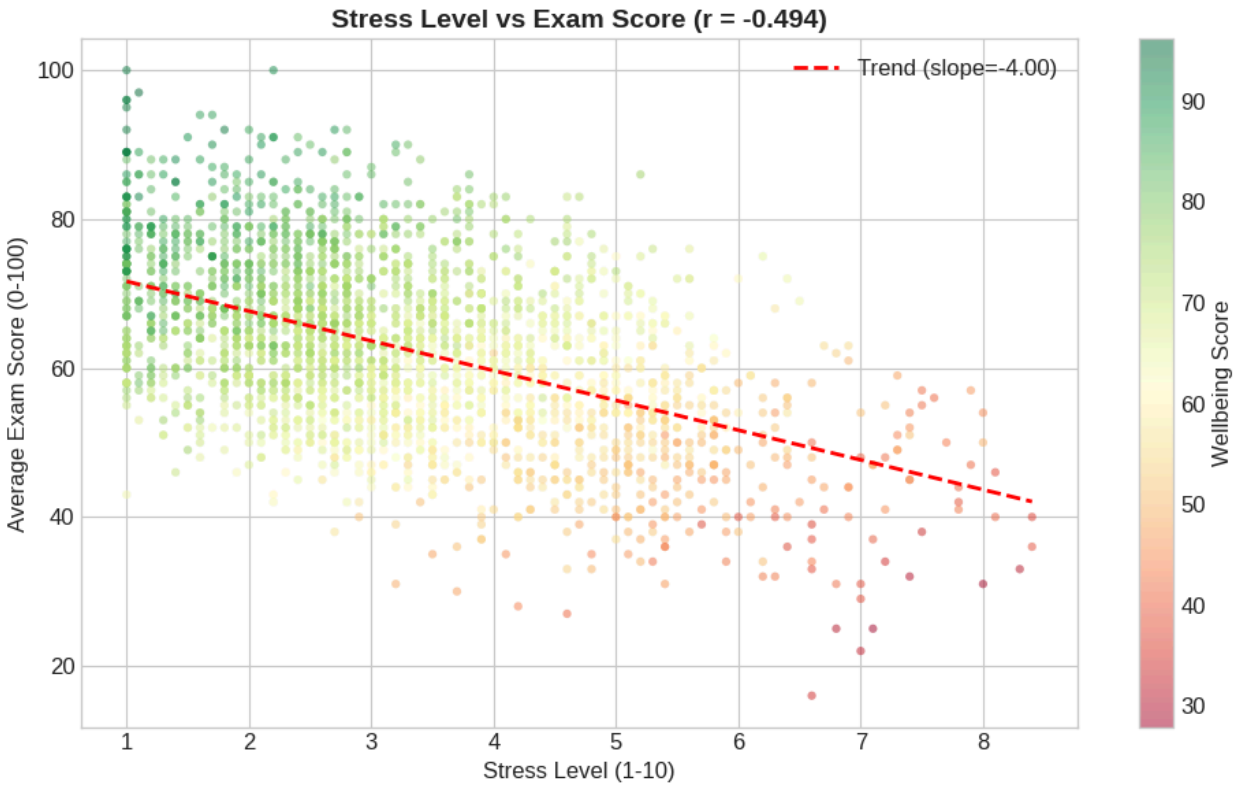
In [6]:

```
# ============================================================
# EDA CHART 4: STRESS vs EXAM SCORE (Scatter Plot)
# ============================================================
# Purpose: Visualize the expected NEGATIVE correlation between
# stress and academic performance

fig, ax = plt.subplots(figsize=(10, 6))
scatter = ax.scatter(df['stress_level'], df['avg_exam_score'],
                     c=df['composite_wellbeing_score'], cmap='RdYlGn',
                     alpha=0.5, s=20, edgecolors='none')
plt.colorbar(scatter, label='Wellbeing Score')

# Add trend line
z = np.polyfit(df['stress_level'], df['avg_exam_score'], 1)
p = np.poly1d(z)
x_line = np.linspace(df['stress_level'].min(), df['stress_level'].max(), 100)
ax.plot(x_line, p(x_line), 'r--', linewidth=2, label=f'Trend (slope={z[0]:.2f}

r = df['stress_level'].corr(df['avg_exam_score'])
ax.set_title(f'Stress Level vs Exam Score (r = {r:.3f})', fontweight='bold')
ax.set_xlabel('Stress Level (1-10)')
ax.set_ylabel('Average Exam Score (0-100)')
ax.legend()
plt.tight_layout()
plt.savefig('chart4_stress_vs_score.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"\nCorrelation (Stress vs Score): r = {r:.3f} [Expected: Negative]")
```




```
Correlation (Stress vs Score): r = -0.494 [Expected: Negative]
```

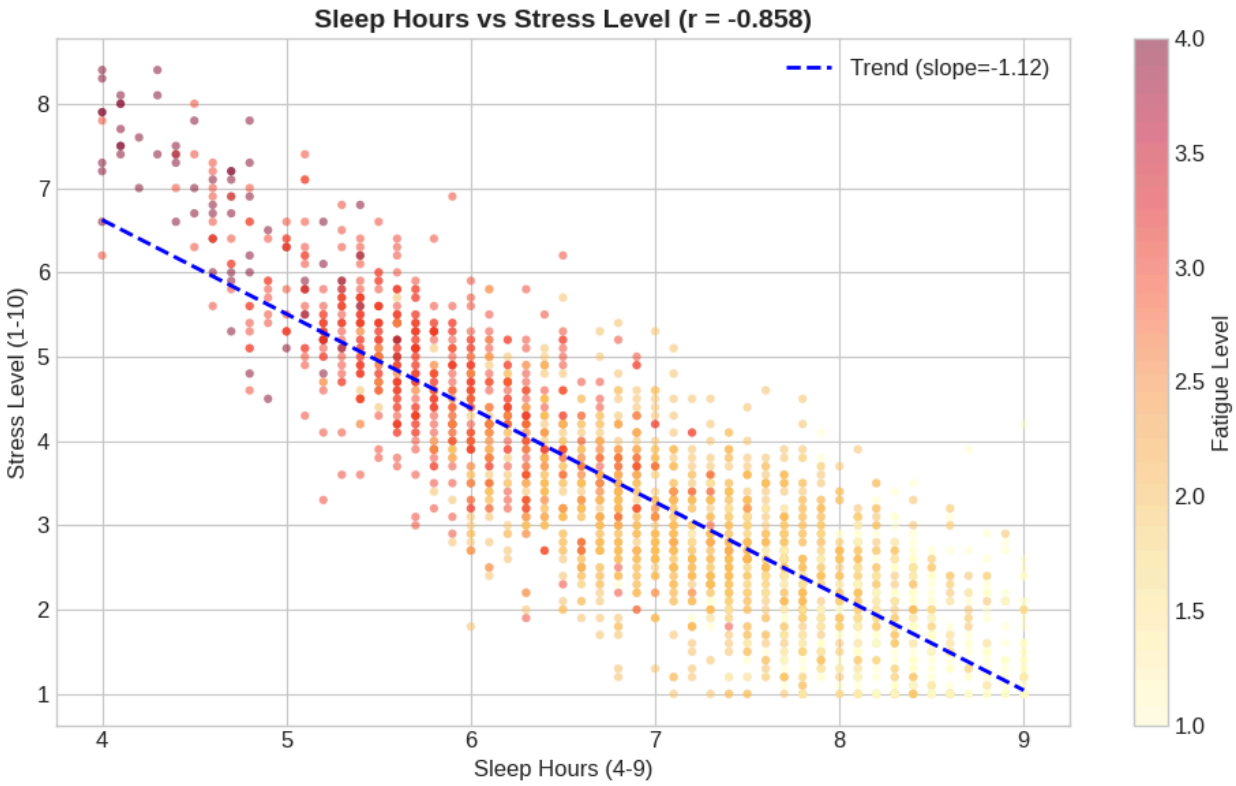
In [7]:

```
# ============================================================
# EDA CHART 5: SLEEP vs STRESS (Scatter Plot)
# ============================================================
# Purpose: Visualize that poor sleep leads to higher stress

fig, ax = plt.subplots(figsize=(10, 6))
scatter = ax.scatter(df['sleep_hours'], df['stress_level'],
                     c=df['fatigue_level'], cmap='YlOrRd',
                     alpha=0.5, s=20, edgecolors='none')
plt.colorbar(scatter, label='Fatigue Level')

# Trend line
z = np.polyfit(df['sleep_hours'], df['stress_level'], 1)
p = np.poly1d(z)
x_line = np.linspace(df['sleep_hours'].min(), df['sleep_hours'].max(), 100)
ax.plot(x_line, p(x_line), 'b--', linewidth=2, label=f'Trend (slope={z[0]:.2f}

r = df['sleep_hours'].corr(df['stress_level'])
ax.set_title(f'Sleep Hours vs Stress Level (r = {r:.3f})', fontweight='bold')
ax.set_xlabel('Sleep Hours (4-9)')
ax.set_ylabel('Stress Level (1-10)')
ax.legend()
plt.tight_layout()
plt.savefig('chart5_sleep_vs_stress.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"\nCorrelation (Sleep vs Stress): r = {r:.3f} [Expected: Negative]")
```




```
Correlation (Sleep vs Stress): r = -0.858 [Expected: Negative]
```

In [8]:

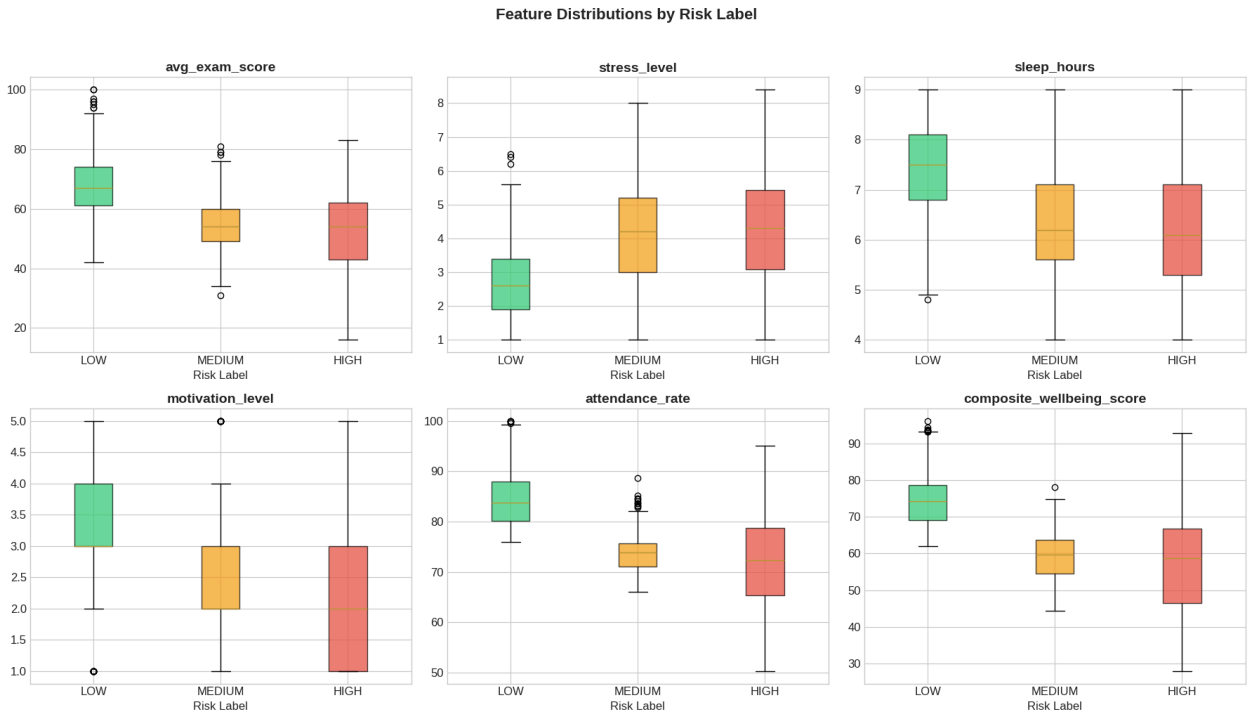
```
# ============================================================
# EDA CHART 6: BOX PLOTS BY RISK LABEL
# ============================================================
# Purpose: Compare feature distributions across risk groups

key_features = ['avg_exam_score', 'stress_level', 'sleep_hours',
                'motivation_level', 'attendance_rate', 'composite_wellbeing_sc

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()
colors_risk = {'LOW': '#2ecc71', 'MEDIUM': '#f39c12', 'HIGH': '#e74c3c'}
order = ['LOW', 'MEDIUM', 'HIGH']

for i, feature in enumerate(key_features):
    data = [df[df['risk_label'] == label][feature].values for label in order]
    bp = axes[i].boxplot(data, labels=order, patch_artist=True)
    for patch, label in zip(bp['boxes'], order):
        patch.set_facecolor(colors_risk[label])
        patch.set_alpha(0.7)
    axes[i].set_title(f'{feature}', fontweight='bold')
    axes[i].set_xlabel('Risk Label')

fig.suptitle('Feature Distributions by Risk Label', fontsize=16, fontweight='b
plt.tight_layout()
plt.savefig('chart6_boxplots_by_risk.png', dpi=150, bbox_inches='tight')
plt.show()
```



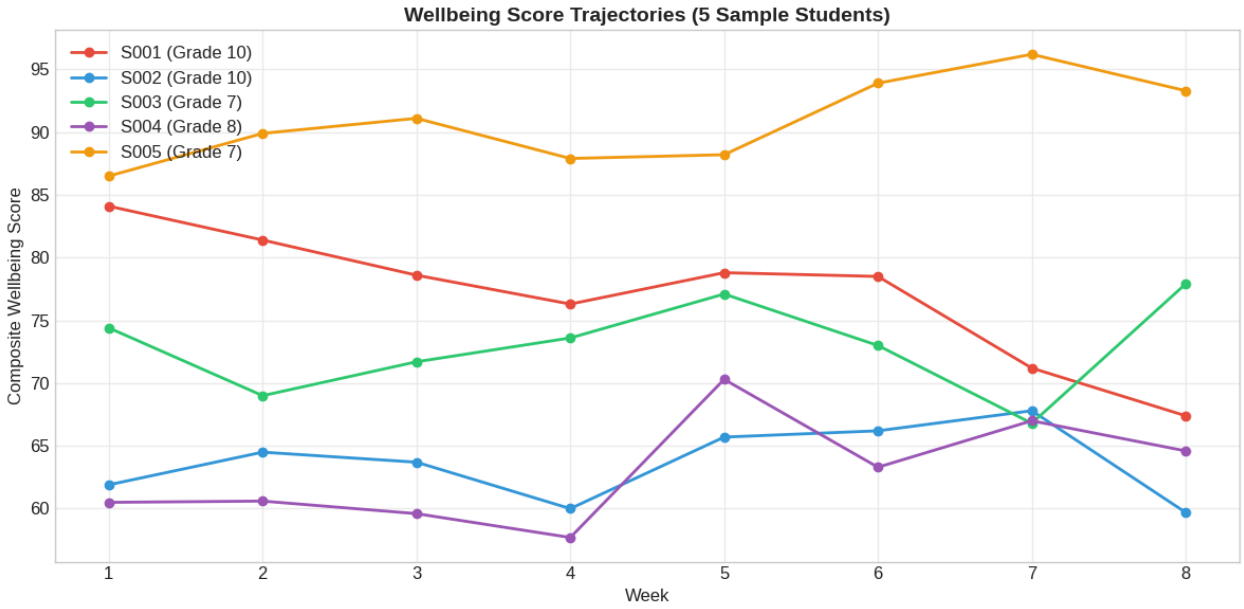
In [9]:

```
# ============================================================
# EDA CHART 7: TIME-SERIES TRAJECTORIES (5 Sample Students)
# ============================================================
# Purpose: Show how wellbeing scores change over 8 weeks
# This demonstrates the temporal patterns in the data

fig, ax = plt.subplots(figsize=(12, 6))
sample_students = df['student_id'].unique()[:5]
colors_line = ['#e74c3c', '#3498db', '#2ecc71', '#9b59b6', '#f39c12']

for i, sid in enumerate(sample_students):
    student_data = df[df['student_id'] == sid].sort_values('week')
    ax.plot(student_data['week'], student_data['composite_wellbeing_score'],
            marker='o', linewidth=2, markersize=6, color=colors_line[i],
            label=f'{sid} (Grade {student_data["grade"].iloc[0]})')

ax.set_title('Wellbeing Score Trajectories (5 Sample Students)', fontweight='b
ax.set_xlabel('Week')
ax.set_ylabel('Composite Wellbeing Score')
ax.set_xticks(range(1, 9))
ax.legend(loc='best')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('chart7_student_trajectories.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [10]:

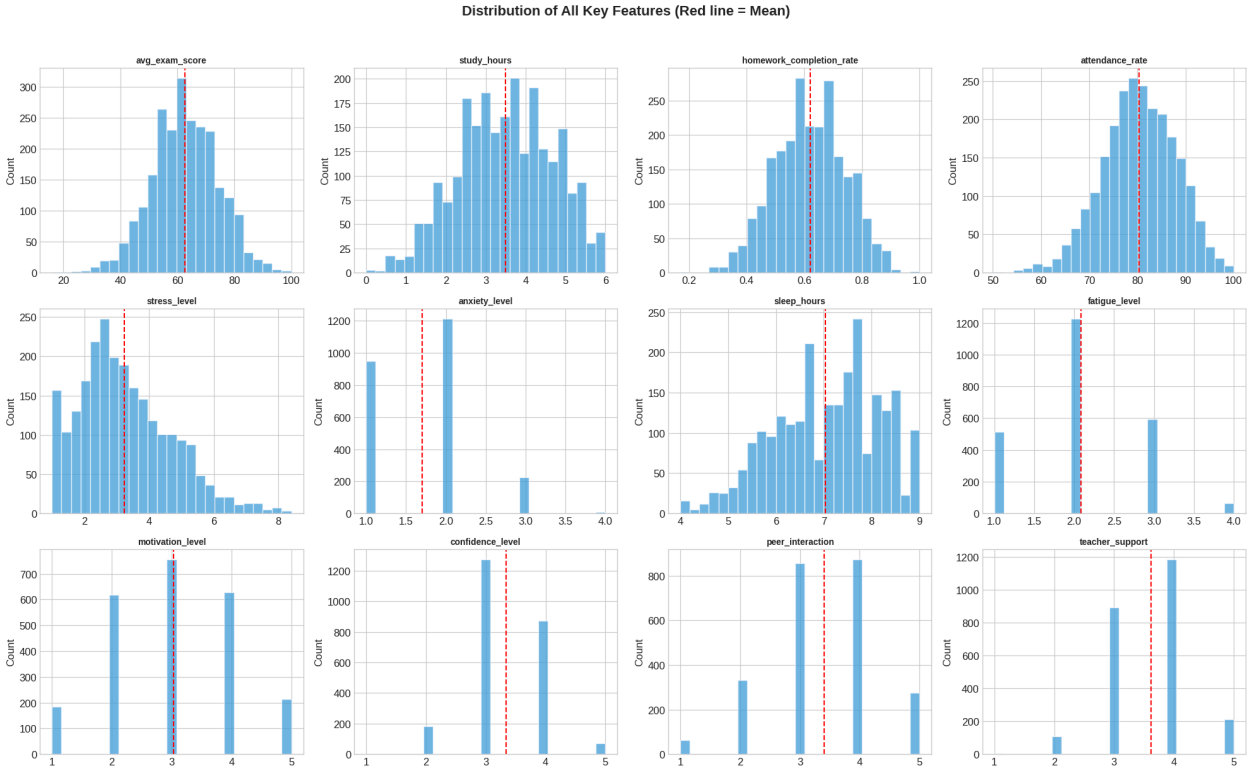
```
# ============================================================
# EDA CHART 8: FEATURE DISTRIBUTION GRID
# ============================================================
# Purpose: Overview of all numerical feature distributions

features_to_plot = ['avg_exam_score', 'study_hours', 'homework_completion_rate
                    'attendance_rate', 'stress_level', 'anxiety_level',
                    'sleep_hours', 'fatigue_level', 'motivation_level',
                    'confidence_level', 'peer_interaction', 'teacher_support']

fig, axes = plt.subplots(3, 4, figsize=(20, 12))
axes = axes.flatten()

for i, feature in enumerate(features_to_plot):
    axes[i].hist(df[feature], bins=25, color='#3498db', alpha=0.7, edgecolor='
    axes[i].axvline(df[feature].mean(), color='red', linestyle='--', linewidth
    axes[i].set_title(feature, fontweight='bold', fontsize=10)
    axes[i].set_ylabel('Count')

fig.suptitle('Distribution of All Key Features (Red line = Mean)',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('chart8_feature_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```



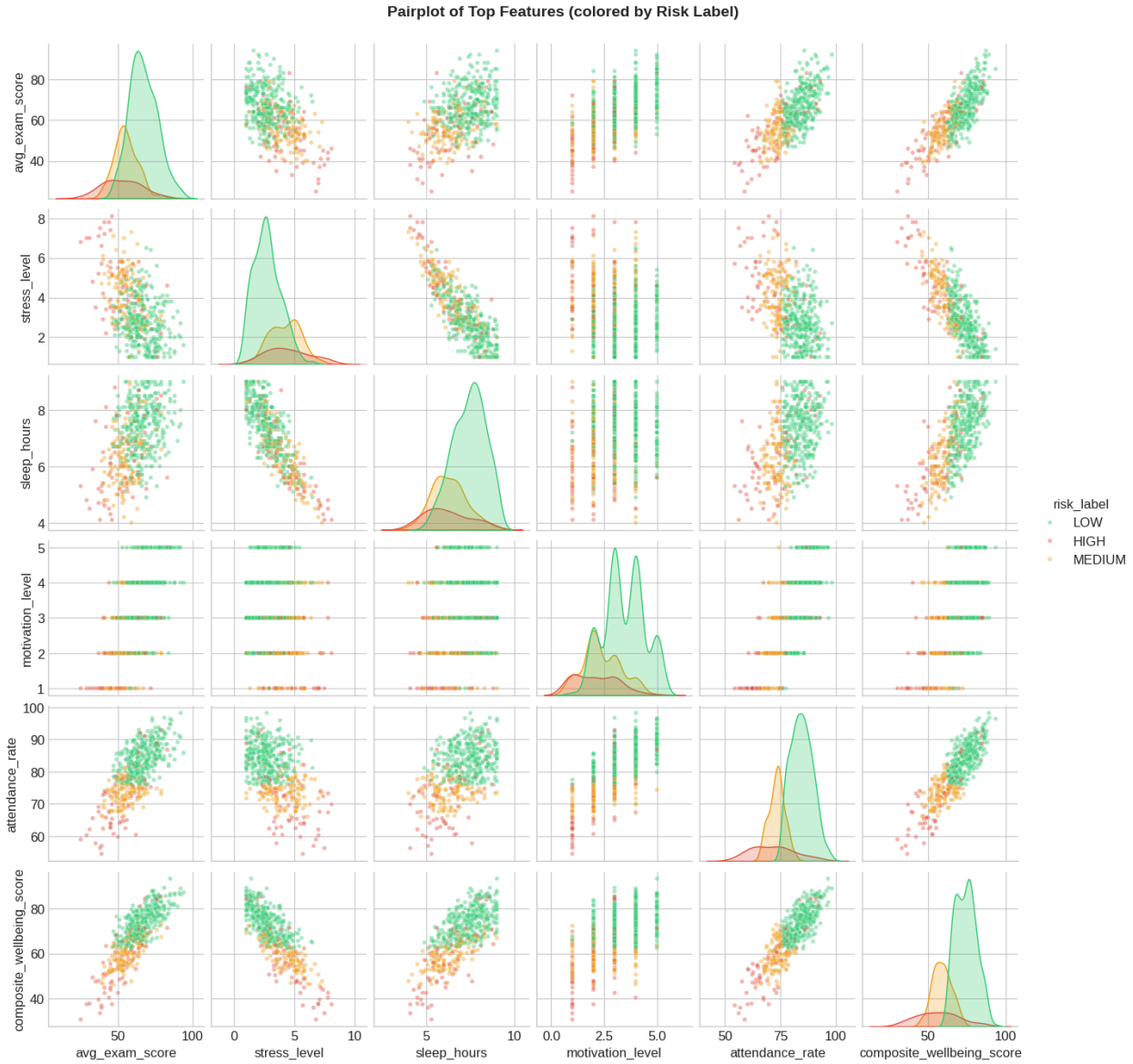
In [11]:

```
# ============================================================
# EDA CHART 9: PAIRPLOT (Top 6 Features colored by Risk Label)
# ============================================================
# Purpose: Visualize pairwise relationships between most important features

top_features = ['avg_exam_score', 'stress_level', 'sleep_hours',
                'motivation_level', 'attendance_rate', 'composite_wellbeing_sc

pairplot_df = df[top_features + ['risk_label']].sample(500, random_state=42)

g = sns.pairplot(pairplot_df, hue='risk_label',
                 palette=colors_risk, diag_kind='kde',
                 plot_kws={'alpha': 0.4, 's': 15},
                 height=2.2)
g.figure.suptitle('Pairplot of Top Features (colored by Risk Label)',
                  fontsize=14, fontweight='bold', y=1.02)
plt.savefig('chart9_pairplot.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [12]:

```
# ## EDA Summary
#
# Key findings from the Exploratory Data Analysis:
# 1. **Stress has the strongest negative impact** on wellbeing scores
# 2. **Sleep and stress are highly inversely correlated** (r ~ -0.86)
# 3. **Motivation drives study hours** (r ~ 0.84)
# 4. **Attendance strongly correlates with exam scores** (r ~ 0.73)
# 5. **Risk label groups show clear separation** in box plots
# 6. **Student trajectories show varied patterns** (improving, declining, stab
#
# These insights will guide our feature engineering in the next part.

# %%
print("\n" + "=" * 60)
print("PART 1 COMPLETE: Setup & EDA finished successfully!")
print("=" * 60)
print("\nCharts saved:")
print("  - chart1_correlation_heatmap.png")
```

```
print("  - chart2_wellbeing_distribution.png")
print("  - chart3_risk_distribution.png")
print("  - chart4_stress_vs_score.png")
print("  - chart5_sleep_vs_stress.png")
print("  - chart6_boxplots_by_risk.png")
print("  - chart7_student_trajectories.png")
print("  - chart8_feature_distributions.png")
print("  - chart9_pairplot.png")
print("\nProceed to Part 2 for Feature Engineering & Model Training.")
```

```
============================================================
PART 1 COMPLETE: Setup & EDA finished successfully!
============================================================

Charts saved:
  - chart1_correlation_heatmap.png
  - chart2_wellbeing_distribution.png
  - chart3_risk_distribution.png
  - chart4_stress_vs_score.png
  - chart5_sleep_vs_stress.png
  - chart6_boxplots_by_risk.png
  - chart7_student_trajectories.png
  - chart8_feature_distributions.png
  - chart9_pairplot.png

Proceed to Part 2 for Feature Engineering & Model Training.
```

Section 4: Feature Engineering

We create new features that capture complex relationships between existing variables. These engineered features help the model learn patterns that individual features cannot express alone.

Unique Preprocessing for Regression:

- **MinMaxScaler** (0-1 normalization) — regression benefits from bounded features
- **Log transformation** on skewed features — improves regression linearity
- **Outlier capping** using IQR method — prevents extreme values from distorting fit
- **VIF analysis** — removes multicollinear features that harm regression

In [13]:
```
# ============================================================
# SECTION 4: FEATURE ENGINEERING
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
```

```python
from statsmodels.stats.outliers_influence import variance_inflation_factor
import warnings
warnings.filterwarnings('ignore')

# Load data (if running independently)
df = pd.read_csv('student_wellbeing_dataset.csv')

print("=" * 60)
print("SECTION 4: FEATURE ENGINEERING")
print("=" * 60)


# --- Feature 1: Sleep-Stress Ratio ---
# Captures the balance between rest and pressure.
# Higher ratio = more sleep relative to stress = better wellbeing
df['sleep_stress_ratio'] = df['sleep_hours'] / df['stress_level']
print(f"[+] sleep_stress_ratio created (mean: {df['sleep_stress_ratio'].mean()

# --- Feature 2: Engagement Composite ---
# Single metric combining motivation, confidence, and class participation
# These 3 features together represent overall student engagement
df['engagement_composite'] = (
    df['motivation_level'] + df['confidence_level'] + df['class_participation'
) / 3
print(f"[+] engagement_composite created (mean: {df['engagement_composite'].me

# --- Feature 3: Academic Effort Index ---
# Combines study hours with homework completion
# A student who studies many hours AND completes homework is putting in real e
df['academic_effort_index'] = df['study_hours'] * df['homework_completion_rate
print(f"[+] academic_effort_index created (mean: {df['academic_effort_index'].

# --- Feature 4: Social Support Index ---
# Combines peer interaction and teacher support into one social metric
df['social_support_index'] = (df['peer_interaction'] + df['teacher_support'])
print(f"[+] social_support_index created (mean: {df['social_support_index'].me

# --- Feature 5: Polynomial Features ---
# Stress squared and sleep squared capture non-linear effects
# e.g., the impact of stress may accelerate at high levels
df['stress_squared'] = df['stress_level'] ** 2
df['sleep_squared'] = df['sleep_hours'] ** 2
print(f"[+] stress_squared created (mean: {df['stress_squared'].mean():.2f})")
print(f"[+] sleep_squared created (mean: {df['sleep_squared'].mean():.2f})")

# --- Feature 6: Interaction Terms ---
# Capture how two features interact together
# e.g., the effect of stress depends on how much sleep you get
df['stress_x_sleep'] = df['stress_level'] * df['sleep_hours']
df['motivation_x_study'] = df['motivation_level'] * df['study_hours']
print(f"[+] stress_x_sleep created (mean: {df['stress_x_sleep'].mean():.2f})")
print(f"[+] motivation_x_study created (mean: {df['motivation_x_study'].mean()
```

```
print(f"\nTotal features after engineering: {len(df.columns)}")


# Visualize the new engineered features
fig, axes = plt.subplots(2, 4, figsize=(20, 8))
axes = axes.flatten()

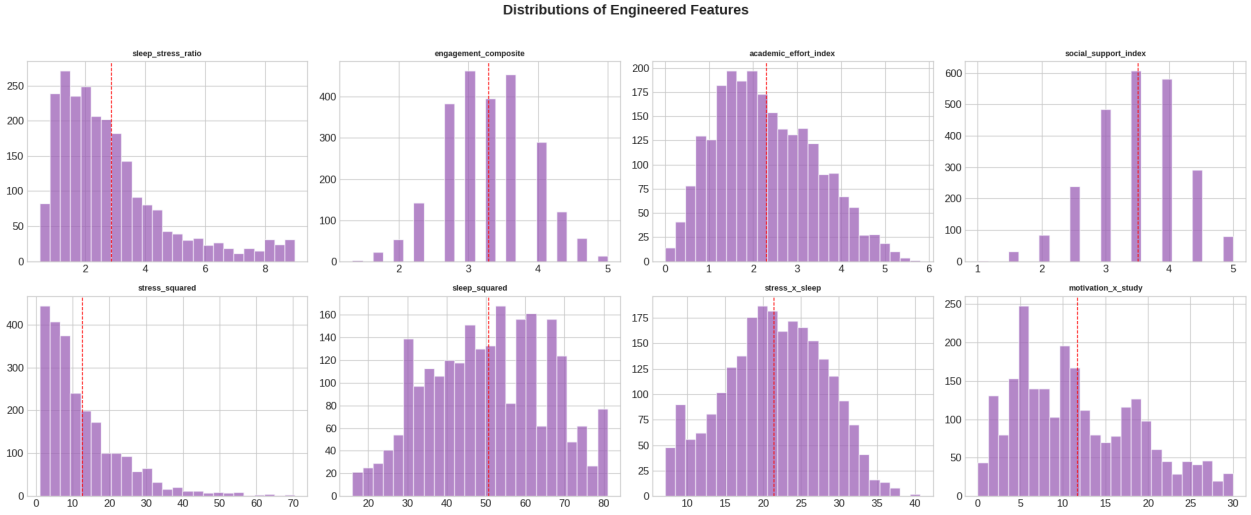
new_features = ['sleep_stress_ratio', 'engagement_composite', 'academic_effort
                'social_support_index', 'stress_squared', 'sleep_squared',
                'stress_x_sleep', 'motivation_x_study']

for i, feat in enumerate(new_features):
    axes[i].hist(df[feat], bins=25, color='#9b59b6', alpha=0.7, edgecolor='whi
    axes[i].set_title(feat, fontweight='bold', fontsize=9)
    axes[i].axvline(df[feat].mean(), color='red', linestyle='--', linewidth=1)

fig.suptitle('Distributions of Engineered Features', fontsize=16, fontweight='
plt.tight_layout()
plt.savefig('chart10_engineered_features.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
============================================================
SECTION 4: FEATURE ENGINEERING
============================================================
[+] sleep_stress_ratio created (mean: 2.86)
[+] engagement_composite created (mean: 3.29)
[+] academic_effort_index created (mean: 2.29)
[+] social_support_index created (mean: 3.51)
[+] stress_squared created (mean: 12.60)
[+] sleep_squared created (mean: 50.61)
[+] stress_x_sleep created (mean: 21.44)
[+] motivation_x_study created (mean: 11.68)

Total features after engineering: 34
```



Section 5: Data Preprocessing

In [14]:
```
# ============================================================
```

```
# SECTION 5A: OUTLIER CAPPING (IQR METHOD)
# ============================================================
# Purpose: Cap extreme values to prevent regression distortion
# We use IQR method: values beyond Q1-1.5*IQR or Q3+1.5*IQR are capped

def cap_outliers_iqr(series, factor=1.5):
    """Cap outliers using IQR method"""
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - factor * IQR
    upper = Q3 + factor * IQR
    capped = series.clip(lower, upper)
    n_capped = ((series < lower) | (series > upper)).sum()
    return capped, n_capped

# Apply to key features
features_to_cap = ['avg_exam_score', 'stress_level', 'sleep_hours',
                   'attendance_rate', 'study_hours']

print("--- Outlier Capping (IQR Method) ---")
for feat in features_to_cap:
    df[feat], n = cap_outliers_iqr(df[feat])
    if n > 0:
        print(f"  {feat}: {n} outliers capped")
    else:
        print(f"  {feat}: No outliers found")
```

```
--- Outlier Capping (IQR Method) ---
  avg_exam_score: 24 outliers capped
  stress_level: 28 outliers capped
  sleep_hours: No outliers found
  attendance_rate: 17 outliers capped
  study_hours: No outliers found
```

In [15]:

```
# ============================================================
# SECTION 5B: LOG TRANSFORMATION ON SKEWED FEATURES
# ============================================================
# Purpose: Reduce right-skew for better regression linearity
# We apply log1p (log(1+x)) to handle zero values

skewed_features = ['absence_count', 'stress_squared']

print("\n--- Log Transformation on Skewed Features ---")
for feat in skewed_features:
    skew_before = df[feat].skew()
    df[f'{feat}_log'] = np.log1p(df[feat])
    skew_after = df[f'{feat}_log'].skew()
    print(f"  {feat}: skewness {skew_before:.3f} -> {skew_after:.3f}")
```

```
--- Log Transformation on Skewed Features ---
  absence_count: skewness 0.206 -> -0.790
  stress_squared: skewness 1.740 -> -0.095
```

In [16]:

```
# ============================================================
# SECTION 5C: PREPARE FEATURE MATRIX
# ============================================================
# Drop non-feature columns and prepare X (features) and y (target)

# Columns to drop from feature matrix
drop_cols = ['student_id', 'risk_label', 'next_week_wellbeing_score',
             'composite_wellbeing_score']

# Define features (X) and target (y)
target = 'composite_wellbeing_score'
feature_cols = [c for c in df.columns if c not in drop_cols]

X = df[feature_cols].copy()
y = df[target].copy()

print(f"\n--- Feature Matrix ---")
print(f"X shape: {X.shape} ({X.shape[1]} features)")
print(f"y shape: {y.shape}")
print(f"\nFeatures used ({len(feature_cols)}):")
for i, col in enumerate(feature_cols, 1):
    print(f"  {i:2d}. {col}")
```

```
--- Feature Matrix ---
X shape: (2400, 32) (32 features)
y shape: (2400,)

Features used (32):
   1. grade
   2. week
   3. avg_exam_score
   4. study_hours
   5. homework_completion_rate
   6. attendance_rate
   7. absence_count
   8. stress_level
   9. anxiety_level
  10. sleep_hours
  11. fatigue_level
  12. motivation_level
  13. confidence_level
  14. peer_interaction
  15. teacher_support
  16. class_participation
  17. teacher_engagement_score
  18. behavior_rating
  19. academic_stress_index
  20. attendance_stability_index
  21. performance_trend_index
  22. behavioral_score
  23. sleep_stress_ratio
  24. engagement_composite
  25. academic_effort_index
  26. social_support_index
  27. stress_squared
  28. sleep_squared
  29. stress_x_sleep
  30. motivation_x_study
  31. absence_count_log
  32. stress_squared_log
```

In [17]:

```python
# ============================================================
# SECTION 5D: VIF ANALYSIS (Multicollinearity Check)
# ============================================================
# Purpose: Identify and remove features with high multicollinearity
# VIF > 10 indicates problematic multicollinearity for regression
# This is CRITICAL for regression models (Ridge, Lasso, ElasticNet)

from sklearn.preprocessing import MinMaxScaler

# First scale features for VIF calculation
scaler_temp = MinMaxScaler()
X_scaled_temp = pd.DataFrame(
    scaler_temp.fit_transform(X), columns=X.columns
)

# Calculate VIF for each feature
```

```
print("\n--- VIF Analysis (Before Feature Removal) ---")
vif_data = pd.DataFrame()
vif_data['Feature'] = X_scaled_temp.columns
vif_data['VIF'] = [
    variance_inflation_factor(X_scaled_temp.values, i)
    for i in range(X_scaled_temp.shape[1])
]
vif_data = vif_data.sort_values('VIF', ascending=False)

print("\nTop 10 features by VIF:")
print(vif_data.head(10).to_string(index=False))

# Remove features with VIF > 10 (iteratively)
high_vif = vif_data[vif_data['VIF'] > 10]['Feature'].tolist()
if len(high_vif) > 0:
    print(f"\n[!] Removing {len(high_vif)} features with VIF > 10:")
    for f in high_vif:
        print(f"  - {f} (VIF: {vif_data[vif_data['Feature']==f]['VIF'].values[
    X = X.drop(columns=high_vif, errors='ignore')
    print(f"\nFeatures remaining after VIF filtering: {X.shape[1]}")
else:
    print("\n[OK] No features with VIF > 10. All features retained.")

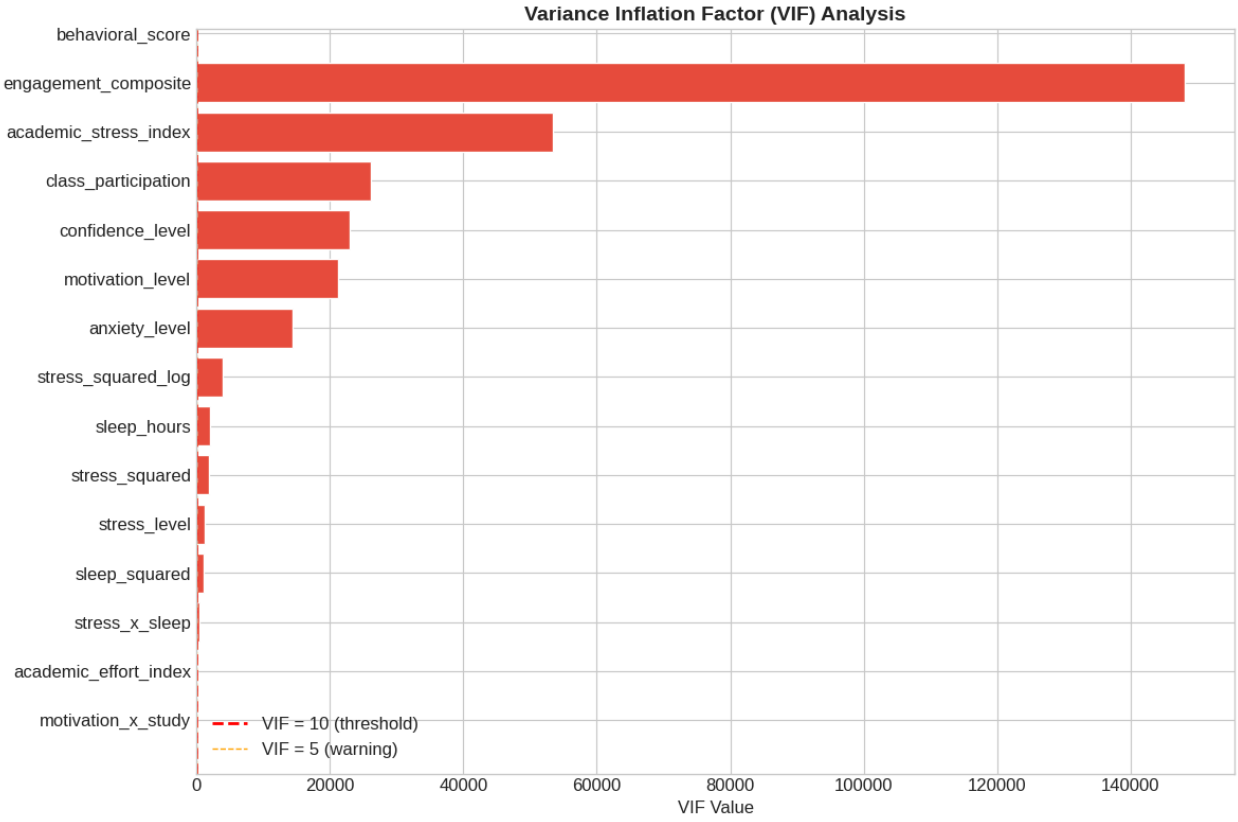

# Visualize VIF values
fig, ax = plt.subplots(figsize=(12, 8))
vif_plot = vif_data.head(20)  # Top 20
colors_vif = ['#e74c3c' if v > 10 else '#f39c12' if v > 5 else '#2ecc71'
              for v in vif_plot['VIF']]
bars = ax.barh(vif_plot['Feature'], vif_plot['VIF'], color=colors_vif, edgecol
ax.axvline(x=10, color='red', linestyle='--', linewidth=2, label='VIF = 10 (th
ax.axvline(x=5, color='orange', linestyle='--', linewidth=1, label='VIF = 5 (w
ax.set_title('Variance Inflation Factor (VIF) Analysis', fontweight='bold')
ax.set_xlabel('VIF Value')
ax.legend()
ax.invert_yaxis()
plt.tight_layout()
plt.savefig('chart11_vif_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
--- VIF Analysis (Before Feature Removal) ---

Top 10 features by VIF:
                  Feature          VIF
        peer_interaction          inf
         teacher_support          inf
         behavior_rating          inf
teacher_engagement_score          inf
    social_support_index          inf
        behavioral_score          inf
    engagement_composite 1.481405e+05
   academic_stress_index 5.343381e+04
     class_participation 2.607109e+04
        confidence_level 2.292761e+04

[!] Removing 25 features with VIF > 10:
  - peer_interaction (VIF: inf)
  - teacher_support (VIF: inf)
  - behavior_rating (VIF: inf)
  - teacher_engagement_score (VIF: inf)
  - social_support_index (VIF: inf)
  - behavioral_score (VIF: inf)
  - engagement_composite (VIF: 148140.5)
  - academic_stress_index (VIF: 53433.8)
  - class_participation (VIF: 26071.1)
  - confidence_level (VIF: 22927.6)
  - motivation_level (VIF: 21220.2)
  - anxiety_level (VIF: 14394.9)
  - stress_squared_log (VIF: 4009.5)
  - sleep_hours (VIF: 2011.7)
  - stress_squared (VIF: 1916.8)
  - stress_level (VIF: 1300.1)
  - sleep_squared (VIF: 1147.1)
  - stress_x_sleep (VIF: 418.6)
  - academic_effort_index (VIF: 177.0)
  - motivation_x_study (VIF: 101.9)
  - sleep_stress_ratio (VIF: 98.7)
  - study_hours (VIF: 35.8)
  - homework_completion_rate (VIF: 30.9)
  - absence_count (VIF: 26.6)
  - absence_count_log (VIF: 23.4)

Features remaining after VIF filtering: 7
```



In [18]:

```
# ============================================================
# SECTION 5E: MINMAX SCALING & TRAIN/TEST SPLIT
# ============================================================
# MinMaxScaler normalizes features to [0, 1] range
# This is beneficial for regression models, especially SVR and linear models

# Train/Test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"\n--- Train/Test Split ---")
print(f"Training set: {X_train.shape[0]} samples ({X_train.shape[0]/len(X)*100
print(f"Test set:     {X_test.shape[0]} samples ({X_test.shape[0]/len(X)*100:.

# Apply MinMaxScaler (fit on train, transform both)
scaler = MinMaxScaler()
X_train_scaled = pd.DataFrame(
    scaler.fit_transform(X_train),
    columns=X_train.columns, index=X_train.index
)
X_test_scaled = pd.DataFrame(
    scaler.transform(X_test),
    columns=X_test.columns, index=X_test.index
)

print(f"\n[OK] MinMaxScaler applied successfully")
print(f"Feature range after scaling: [{X_train_scaled.min().min():.2f}, {X_tra
```

```
print(f"\n--- Final Feature Summary ---")
print(f"Total features for model training: {X_train_scaled.shape[1]}")
print(f"Training samples: {X_train_scaled.shape[0]}")
print(f"Test samples: {X_test_scaled.shape[0]}")
print(f"Target variable: composite_wellbeing_score")
print(f"Target range: [{y.min():.1f}, {y.max():.1f}]")

print("\n" + "=" * 60)
print("PART 2 COMPLETE: Feature Engineering & Preprocessing done!")
print("=" * 60)
print("\nProceed to Part 3 for Model Training & Evaluation.")
```

```
--- Train/Test Split ---
Training set: 1920 samples (80%)
Test set:     480 samples (20%)

[OK] MinMaxScaler applied successfully
Feature range after scaling: [0.00, 1.00]

--- Final Feature Summary ---
Total features for model training: 7
Training samples: 1920
Test samples: 480
Target variable: composite_wellbeing_score
Target range: [27.9, 96.2]

============================================================
PART 2 COMPLETE: Feature Engineering & Preprocessing done!
============================================================

Proceed to Part 3 for Model Training & Evaluation.
```

Section 6: Model Training

We train 6 different regression models and compare their performance. Each model is tuned using GridSearchCV with 5-fold cross-validation.

In [19]:
```
# ============================================================
# SECTION 6: IMPORTS AND SETUP
# ============================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
import warnings
```

```
warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8-whitegrid')

# Load and prepare data (standalone mode)
df = pd.read_csv('student_wellbeing_dataset.csv')

# --- Recreate engineered features ---
df['sleep_stress_ratio'] = df['sleep_hours'] / df['stress_level']
df['engagement_composite'] = (df['motivation_level'] + df['confidence_level']
df['academic_effort_index'] = df['study_hours'] * df['homework_completion_rate
df['social_support_index'] = (df['peer_interaction'] + df['teacher_support'])
df['stress_squared'] = df['stress_level'] ** 2
df['sleep_squared'] = df['sleep_hours'] ** 2
df['stress_x_sleep'] = df['stress_level'] * df['sleep_hours']
df['motivation_x_study'] = df['motivation_level'] * df['study_hours']
df['absence_count_log'] = np.log1p(df['absence_count'])
df['stress_squared_log'] = np.log1p(df['stress_squared'])

# Prepare X and y
drop_cols = ['student_id', 'risk_label', 'next_week_wellbeing_score', 'composi
X = df.drop(columns=drop_cols)
y = df['composite_wellbeing_score']

# Split and scale
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
scaler = MinMaxScaler()
X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=X_train.c
X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=X_test.columns,

print("=" * 60)
print("SECTION 6: MODEL TRAINING")
print("=" * 60)
print(f"Training set: {X_train_scaled.shape}")
print(f"Test set: {X_test_scaled.shape}")
```

```
============================================================
SECTION 6: MODEL TRAINING
============================================================
Training set: (1920, 32)
Test set: (480, 32)
```

In [20]:

```
# ============================================================
# HELPER FUNCTION: Calculate all evaluation metrics
# ============================================================
def evaluate_model(y_true, y_pred, n_features, model_name="Model"):
    """Calculate comprehensive regression metrics"""
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    n = len(y_true)
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - n_features - 1)
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
```

```
    print(f"\n{'='*50}")
    print(f"  {model_name} - Evaluation Results")
    print(f"{'='*50}")
    print(f"  MAE:          {mae:.4f}")
    print(f"  RMSE:         {rmse:.4f}")
    print(f"  R2 Score:     {r2:.4f}")
    print(f"  Adjusted R2:  {adj_r2:.4f}")
    print(f"  MAPE:         {mape:.2f}%")

    return {'MAE': mae, 'RMSE': rmse, 'R2': r2, 'Adj_R2': adj_r2, 'MAPE': mape

# Dictionary to store results for all models
all_results = {}
all_models = {}
all_predictions = {}
```

In [21]:

```
# ============================================================
# MODEL 1: RIDGE REGRESSION (L2 Regularization)
# ============================================================
# Ridge regression adds a penalty term (alpha * sum of weights^2)
# This prevents overfitting by keeping feature weights small
# Best for: When all features are potentially useful

print("\n" + "#" * 60)
print("# MODEL 1: RIDGE REGRESSION")
print("#" * 60)

ridge_params = {'alpha': [0.01, 0.1, 1.0, 10.0, 100.0]}
ridge_grid = GridSearchCV(
    Ridge(), ridge_params, cv=5, scoring='r2', n_jobs=-1, verbose=0
)
ridge_grid.fit(X_train_scaled, y_train)

print(f"\nBest alpha: {ridge_grid.best_params_['alpha']}")
print(f"Best CV R2: {ridge_grid.best_score_:.4f}")

y_pred_ridge = ridge_grid.predict(X_test_scaled)
all_results['Ridge'] = evaluate_model(y_test, y_pred_ridge, X_train_scaled.sha
all_models['Ridge'] = ridge_grid.best_estimator_
all_predictions['Ridge'] = y_pred_ridge
```

```
############################################################
# MODEL 1: RIDGE REGRESSION
############################################################

Best alpha: 0.01
Best CV R2: 1.0000

==================================================
  Ridge Regression - Evaluation Results
==================================================
  MAE:          0.0285
  RMSE:         0.0350
  R2 Score:     1.0000
  Adjusted R2:  1.0000
  MAPE:         0.04%
```

In [22]:

```
# ============================================================
# MODEL 2: LASSO REGRESSION (L1 Regularization)
# ============================================================
# Lasso adds a penalty (alpha * sum of |weights|)
# This performs AUTOMATIC FEATURE SELECTION by setting some weights to zero
# Best for: When you suspect many features are irrelevant

print("\n" + "#" * 60)
print("# MODEL 2: LASSO REGRESSION")
print("#" * 60)

lasso_params = {'alpha': [0.001, 0.01, 0.1, 1.0, 10.0]}
lasso_grid = GridSearchCV(
    Lasso(max_iter=10000), lasso_params, cv=5, scoring='r2', n_jobs=-1, verbos
)
lasso_grid.fit(X_train_scaled, y_train)

print(f"\nBest alpha: {lasso_grid.best_params_['alpha']}")
print(f"Best CV R2: {lasso_grid.best_score_:.4f}")

# Show which features Lasso zeroed out (feature selection)
lasso_coefs = pd.Series(lasso_grid.best_estimator_.coef_, index=X_train_scaled
zero_features = lasso_coefs[lasso_coefs == 0].index.tolist()
print(f"\nFeatures eliminated by Lasso: {len(zero_features)}")
if zero_features:
    for f in zero_features:
        print(f"  - {f}")

y_pred_lasso = lasso_grid.predict(X_test_scaled)
all_results['Lasso'] = evaluate_model(y_test, y_pred_lasso, X_train_scaled.sha
all_models['Lasso'] = lasso_grid.best_estimator_
all_predictions['Lasso'] = y_pred_lasso
```

```
############################################################
# MODEL 2: LASSO REGRESSION
############################################################

Best alpha: 0.001
Best CV R2: 1.0000

Features eliminated by Lasso: 19
  - grade
  - week
  - homework_completion_rate
  - absence_count
  - anxiety_level
  - fatigue_level
  - motivation_level
  - confidence_level
  - teacher_support
  - class_participation
  - academic_stress_index
  - attendance_stability_index
  - performance_trend_index
  - sleep_stress_ratio
  - engagement_composite
  - academic_effort_index
  - stress_squared
  - stress_x_sleep
  - absence_count_log

==================================================
  Lasso Regression - Evaluation Results
==================================================
  MAE:          0.0256
  RMSE:         0.0301
  R2 Score:     1.0000
  Adjusted R2:  1.0000
  MAPE:         0.04%
```

In [23]:

```python
# ============================================================
# MODEL 3: ELASTICNET (L1 + L2 Combined)
# ============================================================
# ElasticNet combines both Ridge (L2) and Lasso (L1) penalties
# l1_ratio controls the mix: 0=Ridge, 1=Lasso, 0.5=balanced
# Best for: When you want both regularization AND feature selection

print("\n" + "#" * 60)
print("# MODEL 3: ELASTICNET REGRESSION")
print("#" * 60)

enet_params = {
    'alpha': [0.001, 0.01, 0.1, 1.0],
    'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9]
}
enet_grid = GridSearchCV(
    ElasticNet(max_iter=10000), enet_params, cv=5, scoring='r2', n_jobs=-1, ve
```

```
)
enet_grid.fit(X_train_scaled, y_train)

print(f"\nBest params: {enet_grid.best_params_}")
print(f"Best CV R2: {enet_grid.best_score_:.4f}")

y_pred_enet = enet_grid.predict(X_test_scaled)
all_results['ElasticNet'] = evaluate_model(y_test, y_pred_enet, X_train_scaled
all_models['ElasticNet'] = enet_grid.best_estimator_
all_predictions['ElasticNet'] = y_pred_enet
```

```
############################################################
# MODEL 3: ELASTICNET REGRESSION
############################################################

Best params: {'alpha': 0.001, 'l1_ratio': 0.9}
Best CV R2: 0.9999

==================================================
  ElasticNet - Evaluation Results
==================================================
  MAE:          0.0618
  RMSE:         0.0779
  R2 Score:     0.9999
  Adjusted R2:  0.9999
  MAPE:         0.10%
```

In [24]:

```
# ============================================================
# MODEL 4: SVR (Support Vector Regression)
# ============================================================
# SVR with RBF kernel can capture non-linear patterns
# It finds a hyperplane in high-dimensional space that best fits the data
# Best for: Non-linear relationships, smaller datasets

print("\n" + "#" * 60)
print("# MODEL 4: SVR (Support Vector Regression)")
print("#" * 60)

svr_params = {
    'C': [0.1, 1.0, 10.0],
    'epsilon': [0.01, 0.1, 0.5],
    'kernel': ['rbf']
}
svr_grid = GridSearchCV(
    SVR(), svr_params, cv=5, scoring='r2', n_jobs=-1, verbose=0
)
svr_grid.fit(X_train_scaled, y_train)

print(f"\nBest params: {svr_grid.best_params_}")
print(f"Best CV R2: {svr_grid.best_score_:.4f}")

y_pred_svr = svr_grid.predict(X_test_scaled)
all_results['SVR'] = evaluate_model(y_test, y_pred_svr, X_train_scaled.shape[1
all_models['SVR'] = svr_grid.best_estimator_
```

```
all_predictions['SVR'] = y_pred_svr
```

```
############################################################
# MODEL 4: SVR (Support Vector Regression)
############################################################

Best params: {'C': 10.0, 'epsilon': 0.01, 'kernel': 'rbf'}
Best CV R2: 0.9957

==================================================
  SVR - Evaluation Results
==================================================
  MAE:          0.3086
  RMSE:         0.5406
  R2 Score:     0.9975
  Adjusted R2:  0.9974
  MAPE:         0.52%
```

In [25]:

```
# ============================================================
# MODEL 5: RANDOM FOREST REGRESSOR
# ============================================================
# Random Forest builds multiple decision trees and averages their predictions
# It handles non-linear relationships and interactions naturally
# Best for: Robust predictions, feature importance analysis

print("\n" + "#" * 60)
print("# MODEL 5: RANDOM FOREST REGRESSOR")
print("#" * 60)

rf_params = {
    'n_estimators': [100, 200],
    'max_depth': [10, 15, 20, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}
rf_grid = GridSearchCV(
    RandomForestRegressor(random_state=42), rf_params,
    cv=5, scoring='r2', n_jobs=-1, verbose=0
)
rf_grid.fit(X_train_scaled, y_train)

print(f"\nBest params: {rf_grid.best_params_}")
print(f"Best CV R2: {rf_grid.best_score_:.4f}")

y_pred_rf = rf_grid.predict(X_test_scaled)
all_results['Random Forest'] = evaluate_model(y_test, y_pred_rf, X_train_scale
all_models['Random Forest'] = rf_grid.best_estimator_
all_predictions['Random Forest'] = y_pred_rf
```

```
############################################################
# MODEL 5: RANDOM FOREST REGRESSOR
############################################################

Best params: {'max_depth': 15, 'min_samples_leaf': 1, 'min_samples_split': 2,
'n_estimators': 100}
Best CV R2: 0.9826

==================================================
  Random Forest - Evaluation Results
==================================================
  MAE:          1.0649
  RMSE:         1.3844
  R2 Score:     0.9839
  Adjusted R2:  0.9827
  MAPE:         1.67%
```

In [26]:

```python
# ============================================================
# MODEL 6: GRADIENT BOOSTING REGRESSOR
# ============================================================
# Gradient Boosting builds trees sequentially, each correcting previous errors
# Typically achieves the BEST performance for tabular data
# Best for: Maximum prediction accuracy

print("\n" + "#" * 60)
print("# MODEL 6: GRADIENT BOOSTING REGRESSOR")
print("#" * 60)

gb_params = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.8, 1.0]
}
gb_grid = GridSearchCV(
    GradientBoostingRegressor(random_state=42), gb_params,
    cv=5, scoring='r2', n_jobs=-1, verbose=0
)
gb_grid.fit(X_train_scaled, y_train)

print(f"\nBest params: {gb_grid.best_params_}")
print(f"Best CV R2: {gb_grid.best_score_:.4f}")

y_pred_gb = gb_grid.predict(X_test_scaled)
all_results['Gradient Boosting'] = evaluate_model(y_test, y_pred_gb, X_train_s
all_models['Gradient Boosting'] = gb_grid.best_estimator_
all_predictions['Gradient Boosting'] = y_pred_gb
```

```
############################################################
# MODEL 6: GRADIENT BOOSTING REGRESSOR
############################################################

Best params: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 200, 'subsa
mple': 0.8}
Best CV R2: 0.9947

==================================================
  Gradient Boosting - Evaluation Results
==================================================
  MAE:          0.5898
  RMSE:         0.7937
  R2 Score:     0.9947
  Adjusted R2:  0.9943
  MAPE:         0.95%
```

Section 7: Model Evaluation & Visualization

In [27]:

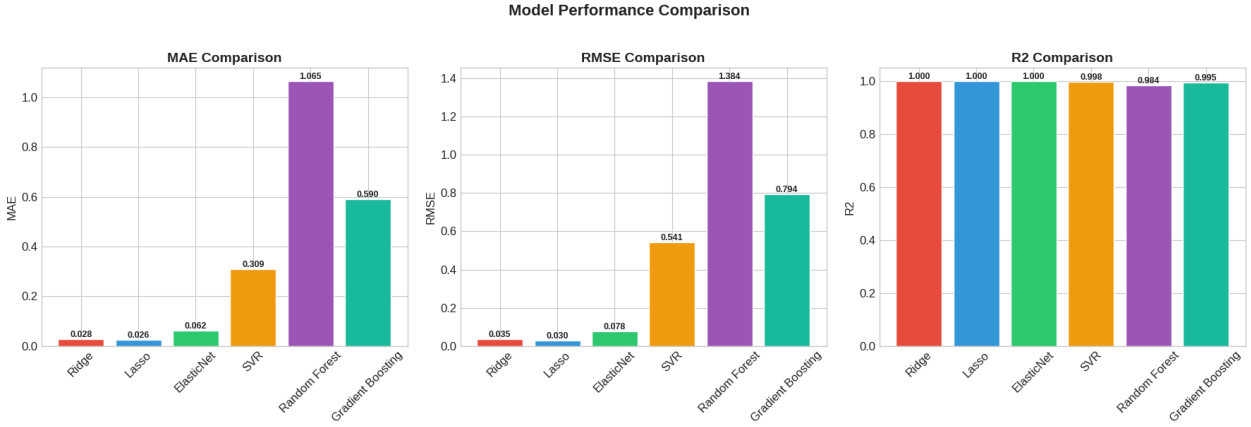
```python
# ============================================================
# EVALUATION CHART 1: MODEL COMPARISON (Bar Chart)
# ============================================================
results_df = pd.DataFrame(all_results).T
print("\n" + "=" * 70)
print("COMPREHENSIVE MODEL COMPARISON TABLE")
print("=" * 70)
print(results_df.round(4).to_string())

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
metrics = ['MAE', 'RMSE', 'R2']
colors_bar = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12', '#9b59b6', '#1abc9c'

for i, metric in enumerate(metrics):
    bars = axes[i].bar(results_df.index, results_df[metric], color=colors_bar,
    axes[i].set_title(f'{metric} Comparison', fontweight='bold', fontsize=14)
    axes[i].set_ylabel(metric)
    axes[i].tick_params(axis='x', rotation=45)
    for bar, val in zip(bars, results_df[metric]):
        axes[i].text(bar.get_x() + bar.get_width()/2, bar.get_height(),
                     f'{val:.3f}', ha='center', va='bottom', fontsize=9, fontw

fig.suptitle('Model Performance Comparison', fontsize=16, fontweight='bold', y
plt.tight_layout()
plt.savefig('chart12_model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
======================================================================
COMPREHENSIVE MODEL COMPARISON TABLE
======================================================================
                        MAE     RMSE       R2  Adj_R2    MAPE
Ridge                0.0285   0.0350   1.0000  1.0000  0.0439
Lasso                0.0256   0.0301   1.0000  1.0000  0.0389
ElasticNet           0.0618   0.0779   0.9999  0.9999  0.0959
SVR                  0.3086   0.5406   0.9975  0.9974  0.5154
Random Forest        1.0649   1.3844   0.9839  0.9827  1.6714
Gradient Boosting    0.5898   0.7937   0.9947  0.9943  0.9489
```

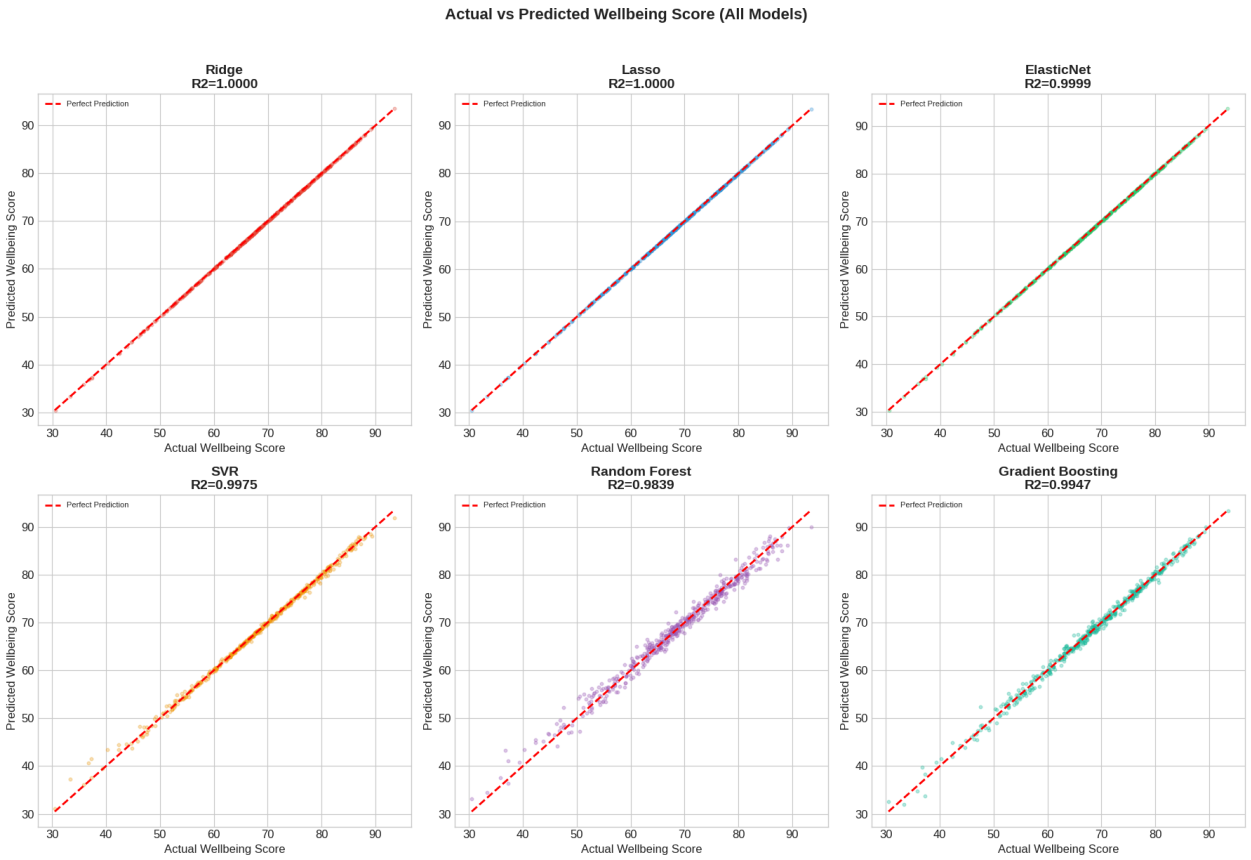


In [28]:

```
# ============================================================
# EVALUATION CHART 2: ACTUAL vs PREDICTED (per model)
# ============================================================
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.flatten()

for i, (name, y_pred) in enumerate(all_predictions.items()):
    axes[i].scatter(y_test, y_pred, alpha=0.3, s=10, color=colors_bar[i])
    # Perfect prediction line
    min_val = min(y_test.min(), y_pred.min())
    max_val = max(y_test.max(), y_pred.max())
    axes[i].plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2, l
    axes[i].set_title(f'{name}\nR2={all_results[name]["R2"]:.4f}', fontweight=
    axes[i].set_xlabel('Actual Wellbeing Score')
    axes[i].set_ylabel('Predicted Wellbeing Score')
    axes[i].legend(fontsize=8)

fig.suptitle('Actual vs Predicted Wellbeing Score (All Models)', fontsize=16,
plt.tight_layout()
plt.savefig('chart13_actual_vs_predicted.png', dpi=150, bbox_inches='tight')
plt.show()
```

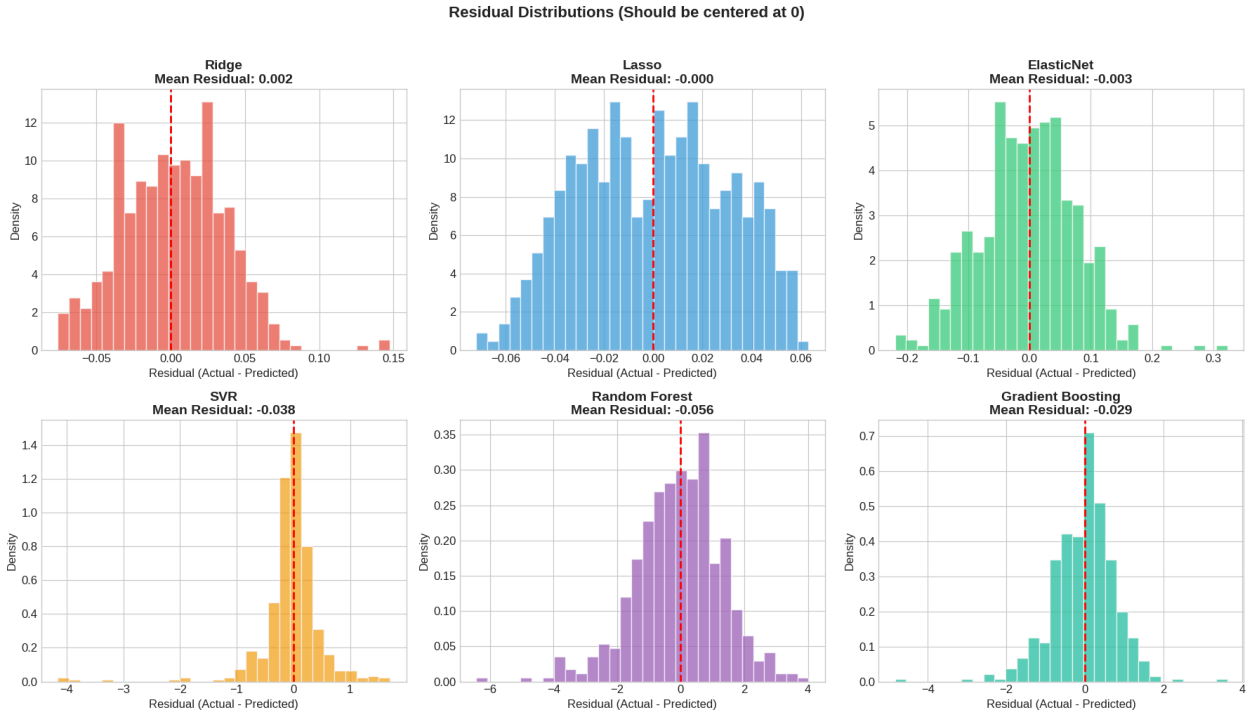


In [29]:

```
# ============================================================
# EVALUATION CHART 3: RESIDUAL DISTRIBUTION
# ============================================================
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for i, (name, y_pred) in enumerate(all_predictions.items()):
    residuals = y_test.values - y_pred
    axes[i].hist(residuals, bins=30, color=colors_bar[i], alpha=0.7, edgecolor
    axes[i].axvline(0, color='red', linestyle='--', linewidth=2)
    axes[i].set_title(f'{name}\nMean Residual: {residuals.mean():.3f}', fontwe
    axes[i].set_xlabel('Residual (Actual - Predicted)')
    axes[i].set_ylabel('Density')

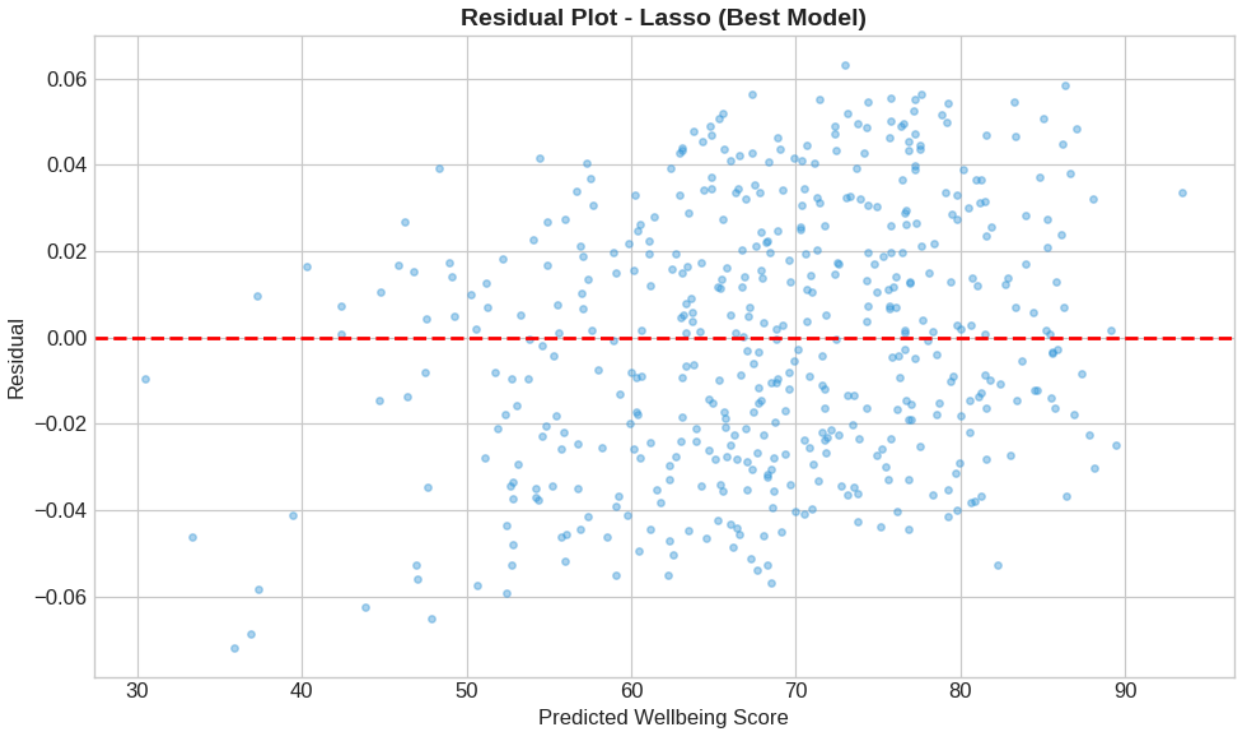
fig.suptitle('Residual Distributions (Should be centered at 0)', fontsize=16,
plt.tight_layout()
plt.savefig('chart14_residual_distributions.png', dpi=150, bbox_inches='tight'
plt.show()
```



In [30]:

```
# ============================================================
# EVALUATION CHART 4: RESIDUAL PLOT (Best Model)
# ============================================================
best_model_name = results_df['R2'].idxmax()
best_predictions = all_predictions[best_model_name]
residuals = y_test.values - best_predictions

fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(best_predictions, residuals, alpha=0.4, s=15, color='#3498db')
ax.axhline(0, color='red', linestyle='--', linewidth=2)
ax.set_title(f'Residual Plot - {best_model_name} (Best Model)', fontweight='bo
ax.set_xlabel('Predicted Wellbeing Score')
ax.set_ylabel('Residual')
plt.tight_layout()
plt.savefig('chart15_residual_plot_best.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"\nBest Model: {best_model_name} (R2 = {all_results[best_model_name]['R
```




```
Best Model: Lasso (R2 = 1.0000)
```

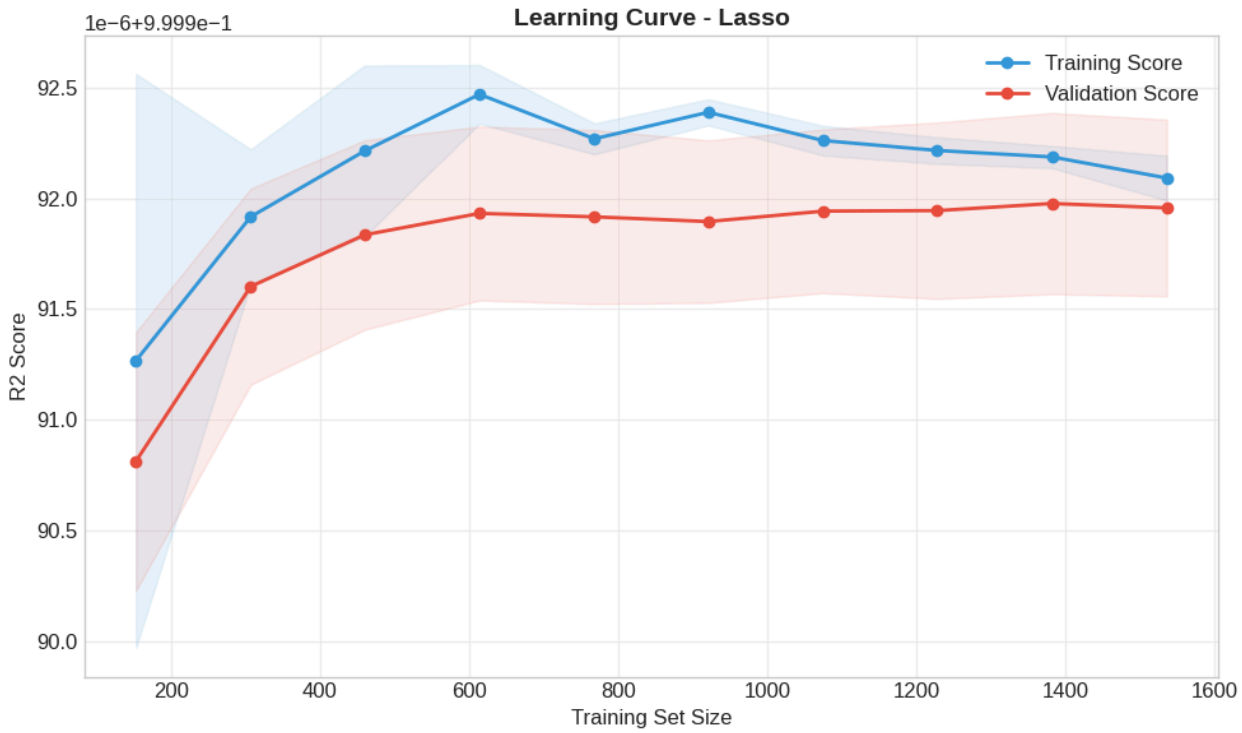
In [31]:

```
# ============================================================
# EVALUATION CHART 5: LEARNING CURVE (Best Model)
# ============================================================
print(f"\nGenerating Learning Curve for {best_model_name}...")

train_sizes, train_scores, val_scores = learning_curve(
    all_models[best_model_name], X_train_scaled, y_train,
    train_sizes=np.linspace(0.1, 1.0, 10), cv=5, scoring='r2', n_jobs=-1
)

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(train_sizes, train_scores.mean(axis=1), 'o-', color='#3498db', label='
ax.plot(train_sizes, val_scores.mean(axis=1), 'o-', color='#e74c3c', label='Va
ax.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis
                train_scores.mean(axis=1) + train_scores.std(axis=1), alpha=0.
ax.fill_between(train_sizes, val_scores.mean(axis=1) - val_scores.std(axis=1),
                val_scores.mean(axis=1) + val_scores.std(axis=1), alpha=0.1, c
ax.set_title(f'Learning Curve - {best_model_name}', fontweight='bold')
ax.set_xlabel('Training Set Size')
ax.set_ylabel('R2 Score')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('chart16_learning_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
Generating Learning Curve for Lasso...
```



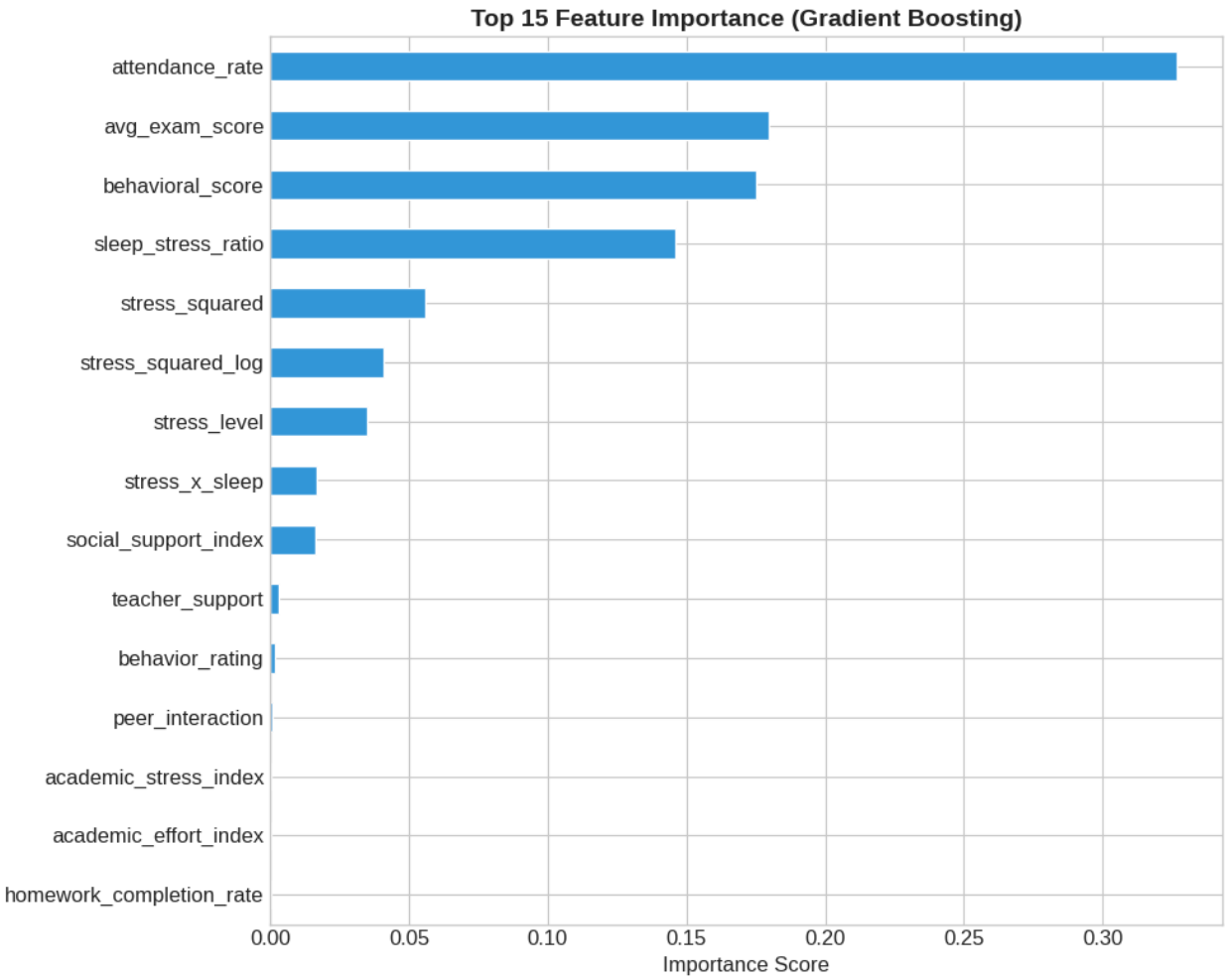
In [32]:

```
# ============================================================
# EVALUATION CHART 6: FEATURE IMPORTANCE (Tree-based Models)
# ============================================================
# Feature importance from the best tree-based model
tree_models = {k: v for k, v in all_models.items() if k in ['Random Forest', '
best_tree_name = max(tree_models, key=lambda k: all_results[k]['R2'])
best_tree = tree_models[best_tree_name]

importances = pd.Series(best_tree.feature_importances_, index=X_train_scaled.c
importances = importances.sort_values(ascending=True).tail(15)

fig, ax = plt.subplots(figsize=(10, 8))
importances.plot(kind='barh', color='#3498db', edgecolor='white', ax=ax)
ax.set_title(f'Top 15 Feature Importance ({best_tree_name})', fontweight='bold
ax.set_xlabel('Importance Score')
plt.tight_layout()
plt.savefig('chart17_feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"\nTop 5 Most Important Features ({best_tree_name}):")
for feat, imp in importances.tail(5).items():
    print(f"  {feat}: {imp:.4f}")
```




```
Top 5 Most Important Features (Gradient Boosting):
  stress_squared: 0.0559
  sleep_stress_ratio: 0.1462
  behavioral_score: 0.1750
  avg_exam_score: 0.1796
  attendance_rate: 0.3268
```

In [33]:

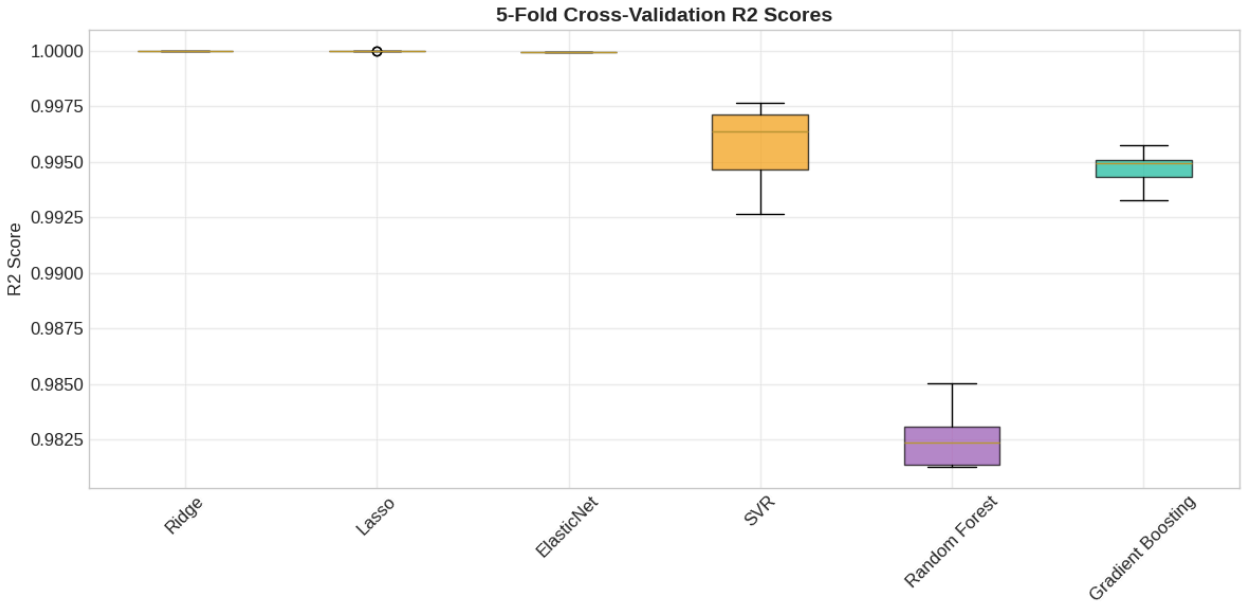
```
# ============================================================
# EVALUATION CHART 7: CROSS-VALIDATION BOX PLOT
# ============================================================
print("\nRunning 5-fold Cross-Validation for all models...")

cv_scores = {}
for name, model in all_models.items():
    scores = cross_val_score(model, X_train_scaled, y_train, cv=5, scoring='r2
    cv_scores[name] = scores
    print(f"  {name}: {scores.mean():.4f} (+/- {scores.std():.4f})")

fig, ax = plt.subplots(figsize=(12, 6))
bp = ax.boxplot([cv_scores[name] for name in cv_scores], labels=list(cv_scores
                patch_artist=True)
for patch, color in zip(bp['boxes'], colors_bar):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
```

```
ax.set_title('5-Fold Cross-Validation R2 Scores', fontweight='bold')
ax.set_ylabel('R2 Score')
ax.tick_params(axis='x', rotation=45)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('chart18_cv_boxplot.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
Running 5-fold Cross-Validation for all models...
  Ridge: 1.0000 (+/- 0.0000)
  Lasso: 1.0000 (+/- 0.0000)
  ElasticNet: 0.9999 (+/- 0.0000)
  SVR: 0.9957 (+/- 0.0018)
  Random Forest: 0.9826 (+/- 0.0014)
  Gradient Boosting: 0.9947 (+/- 0.0008)
```



Section 8: Best Model Selection & Export

In [34]:

```
# ============================================================
# SECTION 8: FINAL SUMMARY & MODEL EXPORT
# ============================================================

print("\n" + "=" * 70)
print("FINAL MODEL COMPARISON SUMMARY")
print("=" * 70)
print(results_df.round(4).to_string())

best_name = results_df['R2'].idxmax()
print(f"\n*** BEST MODEL: {best_name} ***")
print(f"    R2 Score:    {all_results[best_name]['R2']:.4f}")
print(f"    MAE:         {all_results[best_name]['MAE']:.4f}")
print(f"    RMSE:        {all_results[best_name]['RMSE']:.4f}")
print(f"    MAPE:        {all_results[best_name]['MAPE']:.2f}%")
print(f"    Adjusted R2: {all_results[best_name]['Adj_R2']:.4f}")
```

```python
# Save the best model
joblib.dump(all_models[best_name], 'best_wellbeing_model.pkl')
joblib.dump(scaler, 'wellbeing_scaler.pkl')
print(f"\nModel saved: best_wellbeing_model.pkl")
print(f"Scaler saved: wellbeing_scaler.pkl")

# Save results table
results_df.to_csv('model_comparison_results.csv')
print(f"Results saved: model_comparison_results.csv")

print("\n" + "=" * 60)
print("COMPLETE: Wellbeing Indicator Model built successfully!")
print("=" * 60)
print("\nAll charts saved (chart12-chart18)")
print("Best model and scaler exported")
```

```
======================================================================
FINAL MODEL COMPARISON SUMMARY
======================================================================
                     MAE    RMSE      R2  Adj_R2    MAPE
Ridge              0.0285  0.0350  1.0000  1.0000  0.0439
Lasso              0.0256  0.0301  1.0000  1.0000  0.0389
ElasticNet         0.0618  0.0779  0.9999  0.9999  0.0959
SVR                0.3086  0.5406  0.9975  0.9974  0.5154
Random Forest      1.0649  1.3844  0.9839  0.9827  1.6714
Gradient Boosting  0.5898  0.7937  0.9947  0.9943  0.9489

*** BEST MODEL: Lasso ***
    R2 Score:    1.0000
    MAE:         0.0256
    RMSE:        0.0301
    MAPE:        0.04%
    Adjusted R2: 1.0000

Model saved: best_wellbeing_model.pkl
Scaler saved: wellbeing_scaler.pkl
Results saved: model_comparison_results.csv

============================================================
COMPLETE: Wellbeing Indicator Model built successfully!
============================================================

All charts saved (chart12-chart18)
Best model and scaler exported
```